

Playback events & ad events tracking

 Maintainer	 kierandg
 Category	
 Created time	August 17, 2025 11:48 PM

Playback session

Khái niệm

Playback session là một phiên phát **nội dung** trên một **trình phát** (player).

- Khoảng thời gian đại diện cho **playback life-cycle** của một **nội dung chính** (main content) gắn với một **player instance**
- Tính từ lúc bắt đầu play (request to play hay autoplay) cho đến khi chuyển sang content khác hay **kết thúc / thay đổi / khởi tạo** lại player instance, ví dụ: user back ra ngoài, đổi page/screen chứa player, đóng app.
- **Nội dung sẽ bao gồm**: toàn bộ quá trình phát **content chính** và các **asset/video gắn liền** với content chính:
 - Quảng cáo (pre/mid/post-roll) → **KHÔNG ĐƯỢC** thay đổi **playback session**
 - Trailer auto-play trước content, recap intro skip, after-credit ... (nếu có) → **KHÔNG NÊN** thay đổi playback session → **còn tùy cách xử lý hay yêu cầu nghiệp vụ**

Vì khái niệm **playback session** liên quan trực tiếp đến **content** nên cần đi qua các định nghĩa (definition) liên quan đến **content**.

Assets

Digital asset hay **asset** (tạm dịch là tài nguyên/tài sản kỹ thuật số) được hiểu là bất kỳ **tư liệu số (digital material)** nào có giá trị đối với tổ chức → được **ingest** (nhập, đưa vào) vào một hệ thống quản lý như MAM/DAM để tổ chức, lưu trữ, tìm kiếm, phân phối và tái sử dụng hiệu quả.

- Digital material có thể là: văn bản, hình ảnh, âm thanh, video, hoạt hình hoặc các file đa phương tiện khác.
 - Có thể là file video mới quay, bản thiết kế ban đầu, hay một file văn bản chưa được phân loại.
 - Nếu là dữ liệu thô (raw source), chúng có thể được xử lý thêm (transcoding/encoding)
 - Quá trình ingest cũng sẽ tạo ra nhiều định dạng khác nhau, phù hợp cho nhiều mục đích sử dụng.
- Một khi đã được đưa vào hệ thống, một **digital asset** sẽ có thêm các **siêu dữ liệu (metadata)** như ID, tiêu đề, mô tả, phân loại, thông tin bản quyền ...



DAM (Digital Asset Management) tạm dịch là “hệ thống quản lý tài nguyên (hay tài sản) kỹ thuật số”

- Dùng để quản lý một cách có hệ thống tất cả các loại tài sản kỹ thuật số của một tổ chức.
- Quản lý nhiều loại tài nguyên (hay tài sản) số như hình ảnh, video, audio, tài liệu văn bản, đồ họa, thiết kế ...
 - **Hình ảnh:** ảnh sản phẩm, đồ họa, logo.
 - **Video và âm thanh:** như clip quảng cáo, podcast.
 - **Tài liệu:** PDF, Word, PowerPoint.
 - **Thiết kế:** như file design gốc của Adobe (PSD, AI), file CAD.
- Mục đích chính
 - Cung cấp một kho lưu trữ tập trung, cho phép người dùng dễ dàng tìm kiếm, lưu trữ, truy cập
 - Phân phối các tài sản này một cách an toàn và hiệu quả.
 - Hỗ trợ kiểm soát **phiên bản**, **quyền truy cập** và **vòng đời** của tài sản.

Tương tự, **MAM** (media asset management) cũng là hệ thống dùng để quản lý tài sản/tài nguyên cần thiết cho các hoạt động kinh doanh.

- Nhánh con của DAM chuyên biệt hơn.
- Tập trung vào việc quản lý các **tài sản truyền thông** (media assets) tức photography/images, animations, video và audio.

Trong **ngành truyền hình và video** (broadcast/video industry):

- Ranh giới giữa DAM và MAM → bị xóa mờ MAM $\approx$ DAM, vì tài sản chính cần quản lý chủ yếu là **video, audio, hình ảnh** (photography/audio/visual) cùng **metadata đi kèm**.
- MAM sẽ focus hơn vào các metadata → hỗ trợ quy trình sản xuất, chỉnh sửa, phát hành, phân phối nội dung video/âm thanh của ngành truyền hình, video.
- Ví dụ chức năng điển hình:
 - Tự động gắn thẻ (auto tagging) như nguồn capture/input, ngày giờ, định dạng...
 - Tạo ra nhiều **renditions** chuẩn (còn gọi là standard profiles tương ứng với các common video formats và bitrates).

Content

Content là thành phẩm cuối cùng được tạo ra từ việc kết hợp nhiều **assets**, qua quá trình thiết kế, biên tập. Mục đích của content là truyền tải một thông điệp cụ thể đến khán giả (audience) hay người tiêu thụ (consumer).

- **Content** có **giá trị giao tiếp** và phục vụ một **mục đích nhất định**.
- **Assets** là các thành phần, "khối xây dựng" (building block) cơ bản → sử dụng để tạo ra content.
- Sự khác biệt giữa **assets** và **content** nằm ở mục đích và ứng dụng.

Ví dụ:

- Một công ty chuyên thiết kế **poster** (cho phim):
 - Dùng ảnh, logo, font chữ → đây là các **assets** đầu vào.
 - Poster hoàn chỉnh → là **content**, vì là sản phẩm cuối cùng (final output/product), sẵn sàng phân phối.
- Khi một công ty phim **ingest poster** này vào hệ thống MAM/DAM của mình:
 - Poster được gắn metadata (ví dụ: phân loại “poster chính”, định dạng “PNG”) → có **asset ID**.
 - Poster lúc này trở thành **media asset** trong hệ thống, có thể quản lý, lưu trữ, tái sử dụng.

- Ví dụ, poster có thể được dùng trong banner trên website, listing phim dạng thumbnail / artwork hay lên bài trên social → dùng cho nhiều content khác nhau
- Công ty phim cũng sử dụng poster (asset) cùng với các asset khác (video, phụ đề, trailer):
 - Quá trình biên tập, lên bài → tạo ra **bộ phim** (movie) hoàn chỉnh → publish → transcoding/packaging → CDN → phân phối
 - **Bộ phim hoàn chỉnh** là **content** → là sản phẩm cuối cùng được phân phối đến khán giả.

Content và playlists

Theo **định nghĩa của content** ở trên, **playlist** hoàn toàn đủ điều kiện để được xem là một **content** vì:

- Sản phẩm cuối cùng, hoàn thiện được chọn lọc, tổ chức (curated) để phân phối đến khán giả
- Playlist được tạo ra từ các **media asset** sẵn có (như video, âm thanh, poster)

Tuy nhiên, trong lĩnh vực video analytics, **“video content”** thường chỉ được định nghĩa là **mức đơn vị nhỏ nhất có thể play độc lập**.

Ví dụ:

- **Phim lẻ (movie)** → tương ứng **1 content ID** duy nhất.
- **Phim bộ (series)** → không phải **1 content ID**, mà được tổ chức như sau:
 - **Series ID** (level cao, định danh toàn bộ bộ phim).
 - **Season ID** (nếu có, dùng để nhóm các tập theo mùa).
 - **Cuối cùng là Episode** mới chính là **content ID** trong analytics.
- Tương tự **playlists** chỉ được xem là containers cho **tracks** (audio) hay video/episodes dùng để tổ chức (organize) content.
- **Trailer** → được xử lý như một **content độc lập** do **có thể play độc lập**, mặc dù liên kết logic tới movie/series.

Định nghĩa playback session

Playback session là một phiên phát media (session) mà user thực hiện các thao tác phát (playback) trên một **video/media player** (cùng player instance) để phát **một content chính** (video, livestream, clip).

- Session = khoảng thời gian đại diện cho **life-cycle** của **một content chính** gắn với một **player instance**
- Tất cả các sự kiện phát sinh khi user tương tác với player (ví dụ: play, pause, seek, buffering, ad play...) trong phiên này → đều được liên kết với cùng một **playback session ID duy nhất**.
 - **Bắt đầu:** khi player được khởi tạo để phát **một content chính cụ thể** (video, livestream, clip) tương ứng một **content ID**
 - **Kết thúc:** session kết thúc khi người dùng đóng player, chuyển screen / page khác, hoặc chuyển sang **content chính khác** (reload, đổi video, đổi channel).
- Nếu trên một màn hình / page (của website) có **N video player** độc lập, thì mỗi player (thực hiện playback) và một content sẽ có **playback session riêng biệt** (N session ID khác nhau).
- Nếu content đã playback và complete (lần đầu tiên) → user thực hiện **restart / phát lại từ đầu** cùng content → xem như **playback session mới**
 - Xem là một **playback session mới** → ví dụ F5 để **refresh page hiện tại** với website, hay click button **“Xem lại từ đầu”**
 - Trường hợp này xem như khởi tạo player → tạo **player instance mới**
- Trường hợp “Up next” hay recommendation → xảy ra khi video hiện tại kết thúc và tự động chuyển sang video tiếp theo → **playback nội dung khác**
 - **Playback session cũ** kết thúc khi content chính trước đó complete.
 - **Playback session mới** được khởi tạo cho content chính kế tiếp.
- Playlist được coi là một **container**, là **tập hợp content** → auto-play video tiếp theo (dù không reload player) → **playback nội dung khác**

- Trong trường hợp **xoay màn hình (orientation change)**
 - Trường hợp **xoay màn hình** (portrait ↔ landscape) hay **vào/thoát fullscreen** → **UI/UX change**
 - Tức cơ bản không thay đổi content chính hay player → playback session ID phải **giữ nguyên không đổi**
 - Android thường sẽ **destroy** activity hiện tại rồi **recreate** một activity mới → player instance có thể bị khởi tạo lại → cần xử lý để session ID **phải được giữ nguyên (retained)**

Trường hợp đặc biệt: trailer và video liên quan (related video)

- Trailer hay các video liên quan (ví dụ: clip hậu trường, phỏng vấn diễn viên) **có thể play độc lập** → **có thể xem là nội dung** nhưng không phải là **nội dung chính**.
- Khi người dùng xem các video này → vẫn sử dụng cùng một **player instance** trong cùng một page/screen với nội dung chính → playback session hiện tại **KHÔNG NÊN kết thúc**.
- Các hành động này **NÊN được ghi lại** như một sự kiện trong playback session hiện tại.
- Có thể sử dụng các thuộc tính property hay custom attributes để định danh các video này thay vì dùng `content_id` ví dụ `content_type`.
- Cách này giúp phân biệt chúng với nội dung chính mà không cần tạo một ID phiên phát mới, vì chúng chỉ là một phần của trải nghiệm xem chính, chứ không phải một nội dung độc lập.

Playback session không chỉ là một khoảng thời gian đơn thuần mà còn là một **đơn vị logic** để **tổ chức dữ liệu analytics**, giúp theo dõi và phân tích hành vi xem video của người dùng một cách toàn diện.

Mục tiêu chính của playback session là:

- Gom nhóm toàn bộ các event phát sinh khi người dùng xem video hoặc quảng cáo trên một player nhất định.
- Giúp ngăn việc ghi log các events một cách rời rạc, không hợp lý.
- **Để phân tích hiệu quả** → có thể dễ dàng phân tích toàn bộ quá trình xem của người dùng đối với một nội dung chính, từ thời gian tải video, thời lượng xem, cho đến các tương tác với quảng cáo.



Tương tự, **CTA-5004 – Common Media Client Data (CMCD)**, media client cũng cần gửi dữ liệu session giúp:

- Tổng hợp nhiều dòng log rời rạc thành một session người dùng duy nhất nhờ **session identifier** (tức key `sid` trong CMCD) .
- Phân tích **QoS** cho người dùng cuối, ví dụ: buffer, bitrate, hoặc lỗi kết nối trong toàn bộ phiên xem.
- Nhóm dữ liệu theo session trên server để đánh giá trải nghiệm người dùng một cách toàn diện.

“Session identification allows thousands of individual server log lines to be interpreted as a single user session, leading to a clearer picture of end-user quality of service. ”

CTA Specification, CTA-5004 (2020-09). *“Web Application Video Ecosystem – Common Media Client Data (CMCD)”* <https://cdn.cta.tech/cta/media/media/resources/standards/pdfs/cta-5004-final.pdf>

Xem thêm <https://techdocs.akamai.com/adaptive-media-delivery/docs/common-media-client-data-amd>

Global properties

Global properties là các property về cơ bản phải (**MUST**) gửi kèm trong event để làm bối cảnh (context) → giúp team phân tích có dữ liệu cơ sở để tính toán hợp nhất, phân chia nhất là cross platform.

Với **ad events** ngoài các **global properties** dùng chung, cần bổ sung thêm **context riêng** liên quan đến **nội dung** (content), **phiên xem** của user (**viewing session** hay **playback session**), thông tin player và thông tin về chính ad.

- **Playback session context**
 - Phiên phát **nội dung** trên một **trình phát** (player) → **life-cycle** của một **nội dung chính** (main content) gắn với một **player instance**

- **Playback session ID** (`playback_id`)
- **Player instance ID** (`player_instance_id`) nếu cần
- **Nội dung chính** → `content_id` .
- Có phải **auto play** không hay là **user-initiated play** (`is_autoplay`)
- **Viewing session context**
 - **"Viewing session"** thường được định nghĩa theo **khoảng thời gian tương tác liên tục**, mà người dùng dành để xem nội dung chứ không nhất thiết bị giới hạn chỉ 1 nội dung.
 - Gắn với **người dùng** (user level) thay vì **player instance + content**.
 - Nếu user **xem liên tục nhiều nội dung khác nhau** mà **không idle quá lâu**, thì vẫn có thể được tính là **một viewing session**.
 - Ví dụ hết tập 1 chuyển qua tập 2, hoặc autoplay sang video tiếp theo → vẫn trong cùng một **viewing session**
 - Ví dụ: user mở app, vào màn hình TV, xem kênh VTV1 rồi chuyển sang VTV3 → nhiều **playback session**.
 - Do có **screens tracking**, mỗi khi user **enter screen** / **exit screen** đều có log với `screen_id` → xem như **viewing session = playback screen life-cycle**
 - **Viewing / watch session ID** = có thể xem `screen_id` + `playback_id` (mỗi lần vào sẽ khởi tạo **player instance**)
- **Content context** → thông tin video/nội dung chính , duration, genre, rating, series/episode, playlist.
 - **Content ID** (`content_id`) tức video, audio.
 - **Content title** (`content_title`)
 - **Content type** (`content_type`) → `primary` / `related` hay `movie` , `episode` , `clip` , `live` , `teaser` / `trailer` ...
 - **Series ID / Series title** (`series_id` , `series_title`) → nếu là phim bộ.
 - **Season / Episode number** (`season` , `episode`) → nếu applicable.
 - **Playlist ID** (`playlist_id`) → nếu được phát trong playlist.
 - **Genre / Category** (`genre` , `category`).
 - **Rating / Parental rating** (`rating`).
 - **Duration** (`duration`).
 - **Release year** (`release_year`).
 - **Language** (`language`).
 - **Region / Country of origin** (`country`).
 - **Live flag** (`is_live`) → true/false.
 - **Asset version / variant** (`renditions`) → nếu có nhiều bản.
 - `media_type` như video/audio
 - `rendition_id`
 - `video_bitrate`
 - `audio_bitrate`
 - `frame_rate`
 - `resolution`
 - `duration`
 - `codec_video`
 - `codec_audio`
- **Ad/Creative context** → thông tin quảng cáo
 - **Ad ID** (`ad_id`)
 - **Ad sequence / position** (`ad_sequence`) nếu có **ad podding**

- **Ad type** (`ad_type`) ví dụ pre/mid/post
- **Ad title** (`ad_title`)
- **Ad duration** (`ad_duration`)
- **Ad advertiser** (`ad_advertiser`)
- **Ad server** (`ad_server`)
- **Demand source** (`demand_source`) như là **DSP** (Demand-Side Platform), **ad network** hay **direct campaign** (ví dụ → mapping AdSystem metadata trong VAST)
- **Campaign ID** (`campaign_id`)
- **Campaign name** (`campaign_name`)
- **Placement type** (`placement_type`)
- **Ad creative ID** (`creative_id`)
- Thông tin **tracking** nếu cần (optional) như → chủ yếu dùng để debug và đối soát
 - Impression (`impression_url`)
 - Start (`start_tracking_url`)
 - First quartile (`first_quartile_tracking_url`)
 - Midpoint (`midpoint_tracking_url`)
 - Third quartile (`third_quartile_tracking_url`)
 - Complete (`complete_tracking_url`)
- Thông tin UTM parameters cần tracking như `utm_source` , `utm_medium` , `utm_campaign` , `utm_term` , `utm_content`
- **Player context** → nhóm property mô tả **tình trạng và cấu hình quan trọng** của **trình phát** (player instance) tại thời điểm event xảy ra → quan trọng để phân tích hành vi playback, hiệu suất, QoE ...
 - **Player type** (`player_type`) ví dụ HTML5, native app
 - **Player name** (`player_name`) như "ExoPlayer", "AVPlayer/AVFoundation", "HTML5Native", "Shaka" ...
 - **Player version** (`player_version`)
 - **Player instance ID** (`player_instance_id`) → có thể đã bao gồm trong **playback session context**
 - **Encryption settings** (`encryption_settings`) như thông tin DRM
 - Có đang bật ABR hay thông tin ABR config như thế nào (`abr_mode`)
 - **Video rendition** (profile) hiện tại (`video_rendition`)
 - **Audio rendition** (track) hiện tại (`audio_rendition`)
 - **PHT hiện tại** (`playhead_time`)
 - **Current volume** (`volume_level`)
 - **Mute** hay không (`mute_state`)
 - **Fullscreen** hay không (`fullscreen_state`)
 - **Playback rate** (`playback_rate`)
 - **Streaming format** (`streaming_format`) như HLS / DASH và có bật low-latency không LL-HLS
 - Có phải là audio only (`audio_only`)
 - Thông tin **video decoder** và **audio decoder**
 - HW decoder true/false
 - Decoder name
 - Thông tin **surface**
 - Render mode (`render_mode`)
 - Số frame đã drop (`dropped_frames`)

- Có thiết lập autoplay không (`autoplay_setting`), tức auto-start playback
- Có thiết lập preload không (`preload_setting`)
- Có thiết lập looping không (`loop_setting`)
- Buffer length (`buffer_length`)

Các nhóm properties bắt buộc (áp dụng cho cả **ad events** và **in-app events** khác):

- **User context**
- **Session context** (app)
- **Device context**
- **Network context** → tùy theo ngữ cảnh **contextual SHOULD**
- **Time context**
- **App context**
- **Environment context**

Playback events

Player-level events

Tất cả các **events ở mức player-level** và **streaming-level** đều **PHẢI** được report, vì đây là nguồn dữ liệu chính để hiểu hành vi người dùng và chất lượng trải nghiệm xem.

Các event này phản ánh chi tiết **toàn bộ vòng đời phát lại** (playback session context) của một nội dung trong player instance.

Player-level events bao gồm:

- **Life-cycle events**: start, pause, resume, stop, end.
- **Seek events**: seek start, seek complete.
- **Buffering/rebuffering events**: buffering/rebuffering start, buffering/rebuffering end.
- **Error events**: error occurred, recover, retry.
- **Quality events**: bitrate change, resolution change, dropped frames.
- **Ad-related events**: ad start, ad complete, ad skip → xem chi tiết phần **ad events**

Việc thu thập đầy đủ các playback events giúp:

- **Đo lường hành vi người dùng**: tần suất pause, seek, drop-off points.
- **Phân tích QoE và performance**: thời gian buffer, error rate, adaptive bitrate switching, ad performance.
- **Tối ưu hóa nội dung & đề xuất**: xác định nội dung có tỷ lệ skip cao, điểm dừng phổ biến.
- **Đảm bảo tracking xuyên suốt**: phân biệt rõ giữa content chính, ad, trailer, hoặc asset liên quan.



Xem thêm

- MUX playback events <https://www.mux.com/docs/guides/mux-data-playback-events>
- Firework (Flutter SDK) <https://docs.firework.com/firework-for-developers/flutter-sdk/integration-guide-v2/analytics>
- Adobe Analytics event types <https://experienceleague.adobe.com/en/docs/media-analytics/using/implementation/analytics-only/streaming-media-apis/mc-api-event-types>
- NPAW's video analytics plugin integration process <https://npaw.com/blog/demystifying-analytics-plugin-integrations-for-video-providers/>
- Brightcove player events <https://player.support.brightcove.com/coding-topics/player-events.html>
- THEOPlayer (THEO Technologies giờ thuộc Dolby OptiView) <https://optiview.dolby.com/docs/theoplayer/v7/api-reference/ios/Player%20Events.html>
- JW Player (JWP)
 - Web <https://docs.jwplayer.com/players/docs/jw8-player-events-reference> <https://developer-tools.jwplayer.com/player-event-inspector>
- Kaltura player events
 - Web <https://developer.kaltura.com/player/web/player-events-web> → cơ bản là từ https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Events#media
 - Android <https://developer.kaltura.com/player/android/player-events-android>
 - iOS <https://developer.kaltura.com/player/ios/player-events-ios>

Về cơ bản, **không phải tất cả** các player **đều hỗ trợ đầy đủ** các events như đã nói.

- Có sự khác biệt lớn giữa **native player** (trình phát mặc định) và **commercial player** (trình phát thương mại).
- Native players như (HTML5 `<video>`, iOS `AVPlayer`, Android `ExoPlayer`) chỉ cung cấp các events cơ bản như:
 - **Life-cycle events:**
 - `play` → bắt đầu phát nội dung.
 - `pause` → tạm dừng nội dung.
 - `playing` → nội dung đang trong trạng thái phát (sau khi buffer xong).
 - `ended` → nội dung đã phát xong hoàn toàn.
 - `loadstart` → bắt đầu tải dữ liệu nội dung → sau đó là `durationchange` → biết duration, dimension, variant/track ... có nghĩa là manifest đã parsed rồi
 - `loadedmetadata` → load các metadata như manifest → không phải là static metadata (như thông tin lấy từ API details) → metadata này là của player
 - `loadeddata` → dữ liệu đầu tiên đã được tải, có thể phát → có thể xem là **first frame decoded** (FFD) → có đủ data rồi playback có thể bắt đầu → để chuyển sang `playing` tức **first frame rendered** (FFR)
 - **Seek events:**
 - `seeking` → đang thực hiện tua nội dung.
 - `seeked` → tua đã hoàn thành/dừng tua.
 - **Buffering events:**
 - `waiting` → tạm dừng để tải thêm dữ liệu do bộ đệm cạn (thường gọi là `rebuffering`).
 - `stalled` → không thể lấy dữ liệu/đứng
 - Nhưng hầu hết các sự kiện `waiting` và `stalled` chỉ báo hiệu trình phát đang chờ dữ liệu, **không phân biệt** đó là lần buffer đầu tiên hay buffer lại giữa chừng → cần thêm logic xử lý
 - **Error events:** `error`.

- Ví dụ standard HTML5 video events gồm có `loadstart`, `suspend`, `abort`, `error`, `emptied`, `stalled`, `loadedmetadata`, `loadeddata`, `canplay`, `canplaythrough`, `playing`, `waiting`, `seeking`, `seeked`, `ended`, `durationchange`, `timeupdate`, `progress`, `play`, `pause`, `ratechange`, `volumechange`, `fullscreenchange`, `posterchange`.
- Để có được các sự kiện như yêu cầu (ví dụ `rebuffering` hay `bitrate change`, `dropped frames`), cần sử dụng các trình phát thương mại hoặc thêm các logic.
- Commercial players (JW Player, THEOplayer, Brightcove, v.v.)
 - Tích hợp sẵn các module hay plugin
 - Phân biệt rõ ràng giữa các loại buffering.
 - Tự động report về **bitrate change**, **resolution change**, và **dropped frames**.

Timeline playback

Ngoài các events cơ bản ở mức **player-level** hay **streaming-level** → phải thu thập tất cả các loại **playback events** khác.

Toàn bộ vòng đời playback diễn hình được mô tả như sau:

- **Catalog** là giai đoạn mà nội dung được **liệt kê, sắp xếp và hiển thị** để user có thể tìm kiếm và chọn lựa.
- Sau khi chọn được nội dung ở bước **Catalog** → **Initialization** là giai đoạn khởi tạo → player chuẩn bị các thành phần cần thiết để bắt đầu một **playback session**.

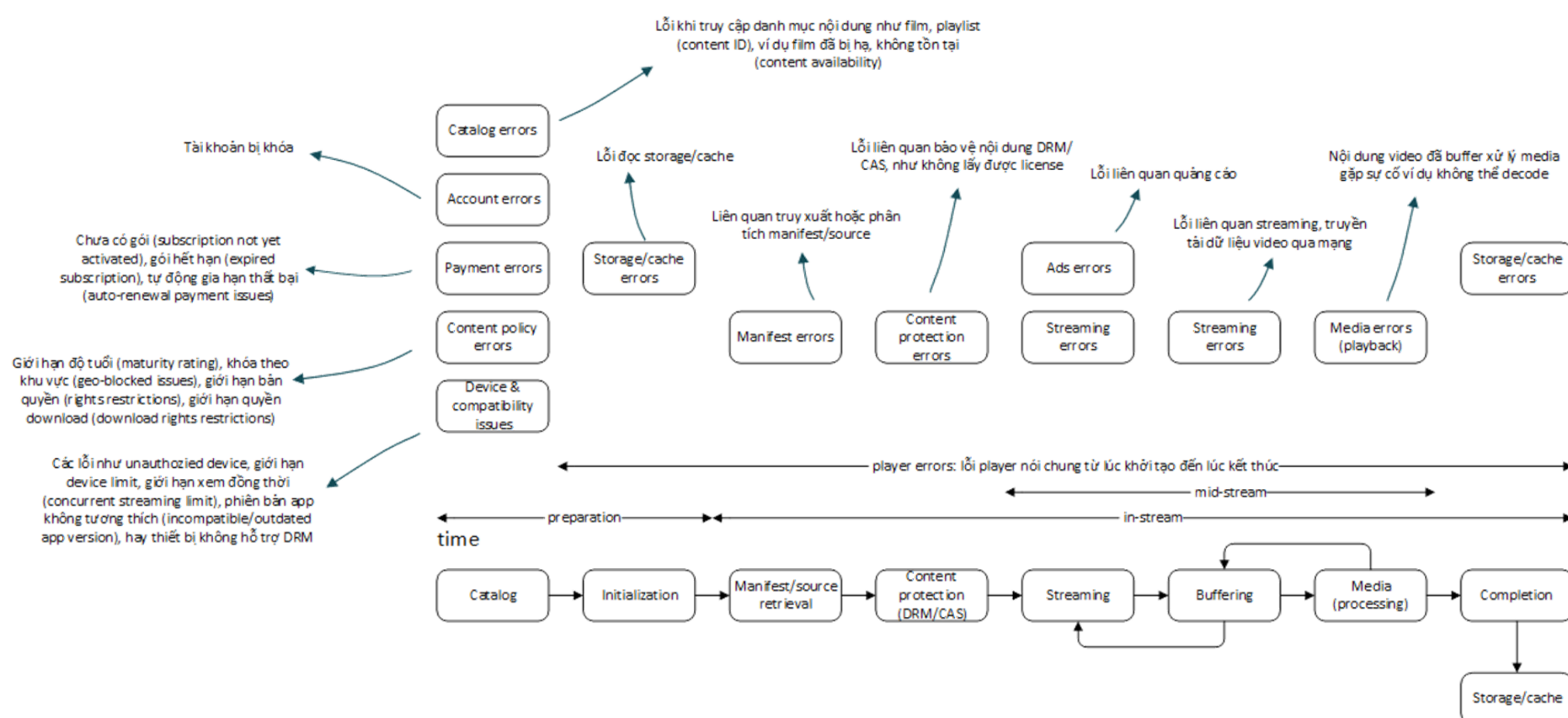
Timeline playback chi tiết

- Đây là chuỗi **events** phản ánh vòng đời tự nhiên của một **playback session**
- Bắt đầu từ sau [Catalog] → **Initialization** → **Manifest retrieval** → **Content protection (DRM/CAS)** → **[Streaming ↔ Buffering ↔ Media (processing)]** → **Finish/Completion** → **Storage/cache**.
- Các events cần report:
 - **Initialization events**: player init, manifest load, content protection request.
 - **Playback start events**: first frame, play.
 - **Playback progression**: progress/heartbeat, pause, resume, seek, buffering start/stop.
 - **Playback end events**: completion, stop, exit.
- Playback session có thể không “kết thúc tự nhiên” (xem hết nội dung) → **dừng/xem lại từ đầu** hay qua **nội dung mới** → có thể dừng do nhiều nguyên nhân lỗi:
 - **Lưu ý**: mỗi khi player khởi tạo lại (re-init manifest) → được coi là **một session mới**.
 - Tất cả các lỗi liên quan cần được report và phân loại

Tất cả các errors **PHẢI được phân loại** kèm thông tin về giai đoạn phát sinh. Hai nhóm giai đoạn lớn là các gian đoạn **chuẩn bị** (preparation) và các giai đoạn **truyền tải và xử lý** (in-stream)

- **Chuẩn bị** (preparation)
 - Lỗi xảy ra ở bước đầu tiên, ngăn không cho video bắt đầu phát.
 - Thường liên quan đến người dùng, tài khoản hoặc thiết bị.
 - **Catalog errors**: Lỗi khi truy cập danh mục. Nội dung user muốn xem không tồn tại hoặc không thể tìm thấy.
 - **Account errors**: Lỗi liên quan đến tài khoản, ví dụ: tài khoản bị khóa.
 - **Payment errors**: Lỗi liên quan đến thanh toán và gói dịch vụ. Có thể user chưa mua gói (subscription), gói đã hết hạn, hoặc có vấn đề khi gia hạn.
 - **Content policy errors**: Lỗi do chính sách nội dung. Ví dụ: user ở khu vực địa lý không được phép xem (geo-blocked), nội dung bị khóa theo độ tuổi, hoặc bị giới hạn quyền xem (rights restrictions).
 - **Device & compatibility issues**: Lỗi liên quan đến thiết bị. Ví dụ: thiết bị không có quyền, vượt quá giới hạn thiết bị được phép xem đồng thời, phiên bản ứng dụng cũ không tương thích, hoặc thiết bị không hỗ trợ công nghệ DRM.
- **Truyền tải và xử lý** (in-stream)

- Đây là các lỗi xảy ra khi video đã bắt đầu phát hoặc đang tải, thường liên quan đến mạng, hệ thống phân phối nội dung hoặc bản thân video.
- **Manifest/source errors:** Lỗi khi trình phát không thể tải hoặc phân tích manifest..
- **Content protection errors:** Lỗi khi trình phát không thể lấy được giấy phép DRM/CAS để giải mã nội dung.
- **Ads errors:** Lỗi xảy ra trong quá trình phát quảng cáo (ví dụ: quảng cáo không tải được, quảng cáo bị lỗi).
- **Streaming errors:** Lỗi do quá trình truyền tải video qua mạng. Tốc độ mạng không ổn định, mất kết nối, hoặc các sự cố khác trên đường truyền.
- **Media errors (playback):** Lỗi xảy ra khi trình phát không thể xử lý hoặc giải mã nội dung video đã tải về. Có thể do tệp video bị hỏng hoặc trình phát gặp vấn đề kỹ thuật khi xử lý định dạng video đó.





- Life-cycle của player và các events cơ bản như sau:



Phân loại	Event	Mô tả
Playback session life-cycle	<code>playback_session_start</code> (optional)	User bắt đầu một phiên playback mới (playback session) → là first intend to play (hay auto play) → <code>play_attempt</code> lần đầu tiên → <code>playback_id</code> kèm player information, <code>player_instance_id</code> (player đã ready to playback) và <code>content_id</code> . · Không tính từ lúc player ready (dù đã xác định player và content) → có thể không phải intend của user (trừ khi autoplay) · Tính các metric khác nhau
	<code>playback_session_end</code> (optional)	Phiên kết thúc (nếu client gửi), thường suy ra từ session timeout ở backend.
	<code>playback_session_pause</code> (optional)	Ghi nhận ví dụ khi ứng dụng chuyển từ foreground → background (nếu ghi nhận được).
	<code>player_initialized</code>	Khởi tạo xong player, player instance được tạo, load các core libraries. Với web player gắn với <code><video></code> tag, tương ứng với <code>loadstart</code> .
Initializations	<code>manifest_request</code> (optional)	
	<code>manifest_response</code> (optional)	
	<code>manifest_loading</code> (optional)	Bắt đầu tải file manifest, chủ yếu dành cho debug
	<code>manifest_loaded</code>	Manifest được tải xong
	<code>manifest_parsed</code>	Đã parsed xong manifest, không lỗi ví dụ sai format → xác định được các variant/track → có thông tin media như duration.
	<code>manifest_error</code>	Lỗi manifest có thể do request failed hay parse error ...
	<code>source_loaded</code>	Nếu không dùng manifest (không streaming với manifest) mà load URL
	<code>metadata_loaded</code> hay <code>player_ready</code>	Player đã load xong metadata → xác định duration, demision, variant/track... (như event <code>loadedmetadata</code> của HTMLMediaElement). Tuy nhiên đây là event thiên về business context vì ngoài manifest có rất nhiều metadata khác ví dụ subtitle hay các timed text file có thể ngoài manifest → chỉ tính các metadata cơ bản đã xác định loaded . Đây là mốc player “ready to start playback” , chứ chưa phải frame displayed → <code>player_ready</code> . · Timeline bar có thể hiển thị (vì đã biết <code>duration</code>) · Thường đã show được nút play (trên player container) → lúc này có thể vẫn chưa FFR (trừ khi pre-loading), thường nút “Play” chỉ hiển thị trên image placeholder (hay poster video) của video canvas .
	<code>first_frame_rendered</code> hay <code>ffr</code>	Đã tải nội dung → content có thể display frame đầu tiên → content displayed Tương ứng web player chuyển sang <code>playing</code> tức playback start (first frame of video) → first frame rendered (FFR) → tức impression
DRM	<code>license_request</code>	
	<code>license_response</code>	
	<code>license_loaded</code>	Player đã load được licence
	<code>license_error</code>	
Playback states	<code>playing</code>	Trong trạng thái hiện đang play (đang phát) → trạng thái playhead chạy, frame liên tục được render. → gửi event <code>playhead</code> / <code>heartbeat</code> định kỳ 10/30/60s để tracking trạng thái
	<code>paused</code>	Trạng thái bị tạm dừng
	<code>completed</code> / <code>ended</code>	Video kết thúc tự nhiên
Buffering state	<code>buffering</code> / <code>buffered</code> hay <code>buffering_started</code> / <code>buffering_ended</code> hay <code>loading_started</code> / <code>loading_ended</code>	Initial buffering/loading → tải hay buffering lần đầu → gửi event <code>heartbeat</code> định kỳ 10/30/60s để tracking trạng thái <code>buffering</code>
	<code>rebuffering</code> / <code>rebuffered</code> hay <code>rebuffering_started</code> / <code>rebuffering_ended</code>	Đang trong trạng thái rebuffering (rebuffering state) → player thiếu buffer đang thực hiện buffering (giữa chừng) → dẫn đến dừng hình (still/freeze), ví dụ hiển thị freeze frame, hay không có video (no video) như màn hình đen → thường sẽ show spinner loading. → gửi event <code>heartbeat</code> định kỳ 10/30/60s để tracking trạng thái <code>rebuffering</code>

Phân loại	Event	Mô tả
Interaction	<code>seeking</code> / <code>seeked</code> hay <code>seek_started</code> / <code>seek_ended</code>	Sự kiện <code>seeking</code> và <code>seeked</code> phản ánh hai giai đoạn khác nhau của quá trình tua và 2 sự kiện này có thể cách xa nhau đáng kể → <code>seeked</code> có thể phát sinh <code>rebuffering</code> tại vị trí mới, decode xong và render được frame → không đơn giản như pause/resume.
	<code>jump</code>	Thực hiện <code>jump</code> ví dụ từ menu hay danh sách → tương tự seek nhưng user không dùng timeline (không kéo/scrub timeline bar)
	<code>play_attempt</code>	User-initiated play (ấn nút Play) với autoplay (player tự chạy khi load) → thực hiện các bước để play Video chưa hẳn là đã show (có thể chỉ có poster) → có thể cần <code>buffering</code> hoặc có thể play ngay do đã thực hiện pre-loading (ví dụ với shorts là dùng pre-loading strategy). Nếu lần đầu tiên (start với playhead = 0) → chuyển qua <code>first_frame</code> → <code>playing</code>. Join time (còn gọi là Video Start Time hoặc Playback Start Latency) → tính từ lúc user yêu cầu playback (viewer initiates playback hay là auto-play) đến FFR. Lần <code>play_attempt</code> đầu tiên (ví dụ có thể failed và user nhấn thêm lần nữa) → có thể xem là start playback session → phát sinh <code>playback_id</code>. - Join time cao → có cảm giác video load chậm, giảm QoE. - Session chỉ thực sự khi user có ý định play (first intend to play) Player Startup Time sẽ tính là khoảng thời gian trước đó, từ lúc player initializing đến khi ready.
	<code>resume_attempt</code>	User chọn resume video (attempt to resume) từ <code>paused</code> → <code>playing</code>
	<code>pause_attempt</code>	User chọn pause video từ <code>playing</code> → <code>paused</code>
	<code>replay_attempt</code>	User chọn replay video (attempt to replay) → tương tự <code>play_attempt</code>
	<code>playback_rate_changed</code> / <code>speed</code>	
	<code>pip_started</code> / <code>pip_ended</code>	→ dùng chung <code>started</code> / <code>ended</code> là convention chung
	<code>cast_started</code> / <code>cast_ended</code>	→ dùng chung <code>started</code> / <code>ended</code> là convention chung
	<code>full_screen_started</code> / <code>full_screen_ended</code>	Fired khi fullscreen mode thay đổi → dùng chung <code>started</code> / <code>ended</code> là convention chung → khởi thay đổi <code>enter</code> / <code>exit</code> hay <code>start</code> / <code>stop</code>
	<code>tapped</code> / <code>clicked</code> (optional)	
	<code>double_tapped</code> (optional)	
Heartbeat / engagement	<code>heartbeat</code>	Gửi event định kỳ để theo dõi playback state → kèm theo cập nhật <code>playhead</code> position. Ngoài playhead các giá trị các thuộc tính (property) khác cần gửi định kỳ bao gồm: <code>throughput_measured</code>, <code>buffer_length</code>, <code>network_latency</code>.
Audio & subtitles tracks	<code>audio_track_loaded</code> (optional)	Tải xong audio track
	<code>audio_track_changed</code>	Chuyển audio track, track đang active thay đổi.
	<code>audio_tracks_updated</code> (optional)	
	<code>text_track_loaded</code> (optional)	Tải xong subtitle
	<code>text_track_changed</code>	Đổi subtitle, track đang active thay đổi.
	<code>text_tracks_updated</code> (optional)	
Volume	<code>muted</code>	
	<code>volume_changed</code>	
QoE & performance (khác)	<code>quality_changed</code>	Thay đổi chất lượng/rendition → adaptive bitrate thay đổi chất lượng video.
	<code>frames_dropped</code>	
Device / environment	<code>orientation_changed</code>	

Phân loại	Event	Mô tả
	<code>app_foregrounded</code> / <code>app_backgrounded</code> (nếu có)	
	<code>network_changed</code>	
Errors & recovery	<code>streaming_error</code>	Lỗi đã xảy ra do không thể tải nội dung (không streaming được nữa) · Ví dụ do network → bị timeout · Bị chặn → ví dụ hạ khẩn cấp → restricted
	<code>streaming_recover</code>	Đã recover → thường liên quan network nên sẽ thực hiện retry policy.
	<code>streaming_retry</code>	Đang thử lại.
	<code>media_error</code>	Đã tải nội dung nhưng không thể play ví dụ không thể decode, ví dụ: · Không play được do không hỗ trợ → not supported · Media error do bị corruption không thể decode → decode · Nội dung encrypted nhưng không thể decrypt → encrypted
	<code>catelog_error</code>	Có thể còn trước cả khi initialize player → chưa có <code>player_instance_id</code> → chưa có <code>playback_id</code> · Content ID không còn nữa → not found · Concurrent streaming limit · Không có quyền xem (playback right)

Error categorization

App có thể dùng nhiều loại player khác nhau (ExoPlayer, AVPlayer, Shaka, Bitmovin...)

- Mỗi player lại có cách **phân loại và trả về error code khác nhau**
- Chưa cần áp chung một hệ thống **error categorization** chi tiết ở client → phức tạp và tốn effort.
- Ở **client side**, chỉ cần đáp ứng mục đích chính là **hiển thị thông báo rõ ràng, dễ hiểu cho user** (ví dụ: “Tài khoản hết hạn”, “Không tải được video”, “Kết nối mạng bị gián đoạn”...).
- Server / analytics layer** → là nơi cần **error categorization** chuẩn hóa

Xử lý qua pipeline như sau:

- Ingestion (thu thập)** → player chỉ cần gửi **mã lỗi gốc** và context đầy đủ (ví dụ: error code -11828 của `AVErrorFileFormatNotRecognized`, hoặc lỗi `MEDIA_ERR_DECODE` từ `HTML5_player`).
- Transformation**
 - Mapping & normalization** → chuẩn hóa tất cả error code của từng loại player về một mã thống nhất và category thống nhất (ví dụ: `ERROR_DECODE` , `ERROR_NETWORK` , `ERROR_DRM`).
 - Enrichment** → bổ sung metadata quan trọng:
 - Giai đoạn xảy ra (preparation vs in-stream)
 - Content ID, session ID
 - User/device info, network info, geo-location
 - Filtering & aggregation:**
 - Lọc các lỗi lặp/không quan trọng (ví dụ retry thành công ngay lập tức)
 - Gom lại thành một **error session report** nếu có nhiều error liên tiếp.
 - Backend/analytics **PHẢI có console management / dashboard**
 - Config & mapping error message → đầu tiên là message hiển thị cho user
 - Cho phép admin định nghĩa hoặc sửa category (Catalog, Account, Payment, DRM/Content protection, Streaming, Buffering, Media, Ads, Storage/Cache...)
 - Có thể **map error code** → **category**

Heartbeat / engagement events

Tương tự như việc đo lường **app session**, trong playback cũng tồn tại:

- **Sự kiện cuối cùng** (last event) không phản ánh chính xác thời điểm người dùng dừng xem.
- Người dùng có thể xem video trong nhiều phút mà **không phát sinh thêm event thao tác nào** (pause, seek, change quality...), trong khi thực tế họ vẫn đang engaged.
- Cần đo lường **watch duration** chính xác thay vì chỉ dựa vào play/pause/ended.
 - **Video on Demand (VOD)**: biết chính xác người dùng xem được bao nhiêu phần nội dung.
 - **Live streaming**: hỗ trợ báo cáo gần real-time về mức độ theo dõi của khán giả.
 - Khi playing cần report **heartbeat events**
- Cần đo lường **Concurrent viewers** (CCU – concurrent users): đo lường số lượng người xem đồng thời tại một thời điểm (nhất là với live streaming).
 - **CCU** = số playback session active tại một thời điểm T.
 - Trong thực tế, không nên và cũng không thể check liên tục từng mili-giây → tính toán theo **interval** (ví dụ mỗi 10 giây, 30 giây, 1 phút).
 - **Interval càng mịn (ngắn)** → số liệu CCU càng gần sát “thực tế” (real-time).

Progress events (thường gửi định kỳ, ví dụ mỗi 10/30s, hoặc tại mốc 25/50/75%)

- Nếu được gửi định kỳ trong khi video đang phát → đóng vai trò như **heartbeat events**.
- Interval của playback heartbeat phải thường **ngắn hơn** so với **app session heartbeat** (ví dụ 30 giây – 1 phút thay vì 3–5 phút)
- Interval thường **yêu cầu dưới 1 phút** → đảm bảo dữ liệu duration mịn và phản ánh chi tiết hành vi xem.

Ad events

Ad cũng có thể được phát như một **media** trong player → trong quá trình **playback ad**, tất cả các **playback events** (như play, pause, buffering/rebuffering, error, complete...) phát sinh đều được tính là một phần **ad events**, **trừ khi có quy định riêng**.

Ngoài các **playback events chung** áp dụng cho cả **content** và **ad**, thì với **ad** còn có thêm **ad-specific events** (ví dụ **impression**, **clicked**) để phục vụ đo lường quảng cáo.



Tất cả các yêu cầu về **playback events** (TRONG PHẦN TRÊN) được định nghĩa cho nội dung đều **PHẢI áp dụng** cho quá trình phát quảng cáo (ad playback).
 Các **playback events** bắt buộc gửi kèm **ad/creative context**.
 Các sự kiện này sẽ được tính là một phần của **ad events** để đảm bảo việc phân tích đầy đủ.

IAB VAST v4 (Video Ad Serving Template v4), **Chapter 3** (VAST Implementation), **Session 3.14.2 <TrackingEvents>** có **định nghĩa** các **sự kiện cần theo dõi** (tracking events) chuẩn cho mỗi creative (Linear hoặc Non-Linear). Các **events** này sẽ kích hoạt khi người dùng tương tác hay quảng cáo đạt đến một mốc nhất định. Ngoài ra, các publisher hoặc advertiser có thể thêm các sự kiện thao theo nhu cầu chuyên biệt (custom events).

Các loại quảng cáo

- **Linear Ad ≠ Non-linear Ad** (của VAST)
 - **Linear Ad** (như **“pre-rolls”**) là những quảng cáo “ngắt ngang” (interrupting ads), bắt buộc người xem phải dừng nội dung chính để xem quảng cáo (ví dụ pre-roll, mid-roll, post-roll).
 - **Non-linear Ad** (như **overlay**) là loại quảng cáo không chiếm toàn bộ màn hình hoặc thời gian phát video chính, chẳng hạn như overlay (lớp phủ), banner, hoặc text ad hiển thị song song (runs concurrently) với nội dung video.
- **Companion Ad** (hay **quảng cáo đi kèm**) là một **định dạng quảng cáo bổ trợ**

- Hiện thị **ngoài khung video player**, thường xuất hiện ở cạnh, dưới hoặc xung quanh video chính đang chạy.
- Ví dụ một **linear video ad** chạy trong player, thì **companion ad** là một banner tĩnh hoặc rich media hiển thị **song song** với video đó.

Phân loại	Event (VAST)	Nhóm	Mô tả
Player Operation Metrics (for use in Linear and NonLinear ads)	mute	Audio controls	User tắt tiếng quảng cáo.
	unmute	// (ditto)	User bật tiếng quảng cáo.
	pause	Playback controls	User tạm dừng quảng cáo.
	resume	//	User tiếp tục (sau khi đã tạm dừng).
	rewind	//	User tua ngược quảng cáo.
	skip	//	User bỏ qua quảng cáo (nếu skippable).
	playerExpand	Display states	User mở rộng player (thay thế fullscreen từ định nghĩa 2014).
	playerCollapse	//	User thu nhỏ player (thay thế exitFullscreen từ định nghĩa 2014).
Linear Ad Metrics	loaded	Ad lifecycle	Đã tải và buffered nội dung QC (creative's media and assets). Có từ VAST v4.1.
	start	Progress / Completion (milestones)	Quảng cáo bắt đầu phát (tương tự creativeView nhưng tập trung vào playback, thông thường là auto-play và muted).
	firstQuartile	//	User xem được hay QC phát liên tục tới 25% thời lượng (ở tốc độ bình thường). Lưu ý đây là event phân loại vào linear ad metrics .
	midpoint	//	User xem được hay QC phát liên tục tới 50% thời lượng (ở tốc độ bình thường).
	thirdQuartile	//	User xem được hay QC phát liên tục tới 75% thời lượng (ở tốc độ bình thường).
	complete	//	User xem được hay QC phát hết 100% thời lượng (ở tốc độ bình thường).
	acceptInvitationLinear	Engagement / Interaction	User chấp nhận lời mời tương tác phần mở rộng tuyến tính.
	timeSpentViewing	//	Thời gian xem quảng cáo (tính bằng giây, hỗ trợ offset hoặc macro).
	otherAdInteraction	//	Các tương tác khác ngoài click thông thường (như hover, click tùy chỉnh, không thay thế các event khác).
	progress-[currentTime] progress-[0-100]%	Progress / Completion (custom)	Phát đến mốc thời gian cụ thể (offset ở định dạng HH:MM:SS hoặc n%, có thể thay thế quartile events).
	creativeView	Engagement / Interaction	Phần quảng cáo (creative) cụ thể nào đã được render đã được xem (riêng biệt với impression). Lưu ý là phân biệt với impression, ví một ad có thể có nhiều creative ví dụ cho từng platform. Event này cho phép server tracking là creative nào được viewed. Đây là event thiên về "exposure" thay vì playback.
	acceptInvitation	//	Người dùng chấp nhận lời mời tương tác (như mở rộng, pause nội dung chính).
Non-linear Ad Metrics	adExpand	Display states	Quảng cáo mở rộng (overlay).
	adCollapse	//	Quảng cáo thu gọn (overlay).
	minimize	//	Quảng cáo bị thu nhỏ (nhỏ hơn trạng thái collapsed).
	close	Playback controls	
	overlayViewDuration	Engagement / Interaction	
	otherAdInteraction	//	

Phân loại	Event (VAST)	Nhóm	Mô tả
Companion Ad Metrics	creativeView	Engagement / Interaction	Companion ad hiển thị (và đã được viewed). Companion ad (như banner hoặc khung quảng cáo hiển thị bên cạnh video) thường được render bằng công nghệ web/browser, nên sự kiện này chỉ đơn giản là creative đã được render lên UI và (ít nhất một phần) hiển thị cho user.

Understanding VAST: The Standard for Video Ad Serving

Version number	Release date	Status
VAST 1.0	August 2008	Deprecated
VAST 2.0	March 2012	Supported, backward-compatible with 4.0+
VAST 3.0	July 2012	Supported, backward-compatible with 4.0+
VAST 4.0	January 2016	Supported
VAST 4.1	November 2018	Supported
VAST 4.2	June 2019	Supported
<u>VAST 4.3</u>	December 2022	Supported (latest major version)

Như vậy các **ad events của VAST** liệt kê ở trên còn thiếu các events liên quan đến phần implementation-layer tức SDK và liên quan đến việc **đo lường performance (QoE hay ad experience)**.

- Các **ad lifecycle events** theo dõi luồng kỹ thuật **ad request** → **manifest load** → **load** → **start** → **complete**.
- Các **issues** và **error events** như phát hiện **ad errors, fallback, timeout**.
- Đo lường **ad QoE / ad experience**.
- Có thể tham khảo
 - Các SDK như **Google IMA**, IAB Open Measurement (OM), các **IMA hay VAST/VPAID plugin** dành cho Video.js
 - Các player như Shaka, THEOplayer, JWP, Kaltura, Brightcove, Wowza Flowplayer
 - Hay các video analytics platform như Adobe Analytics, Conviva, hay MUX



Xem thêm

- Google IMA SDK **ad events**<https://developers.google.com/interactive-media-ads/docs/sdks/html5/client-side/reference/namespace/google.ima.AdEvent>
- IAB Open Measurement (OM) SDK <https://iabtechlab.com/standards/open-measurement-sdk/>
- Adobe Analytics **ad tracking** <https://experienceleague.adobe.com/en/docs/media-analytics/using/tracking/track-ads/track-ads-overview>
- MediaMelon <https://docs.mediamelon.com/mediamelon/smartsight-player-sdk-integration/mediamelon-sdk-events>
- Conviva https://docs.conviva.com/learning-center-files/content/eco/eco_metrics.html
- MUX <https://www.mux.com/docs/guides/mux-data-playback-events>
- Shaka Player <https://shaka-player-demo.appspot.com/docs/api/shaka.ads.AdManager.html#.event:AdMidpointEvent>
- THEOplayer <https://theoplayer.github.io/react-native-theoplayer/api/enums/AdEventType.html>
- JW Player (JWP) <https://docs.jwplayer.com/players/reference/advertising-events>
- Bitmovin Player https://cdn.bitmovin.com/player/web/8/docs/enums/core_events.playerevent.html
- HTML Media events <https://developer.mozilla.org/en-US/docs/Web/API/HTMLMediaElement#events>

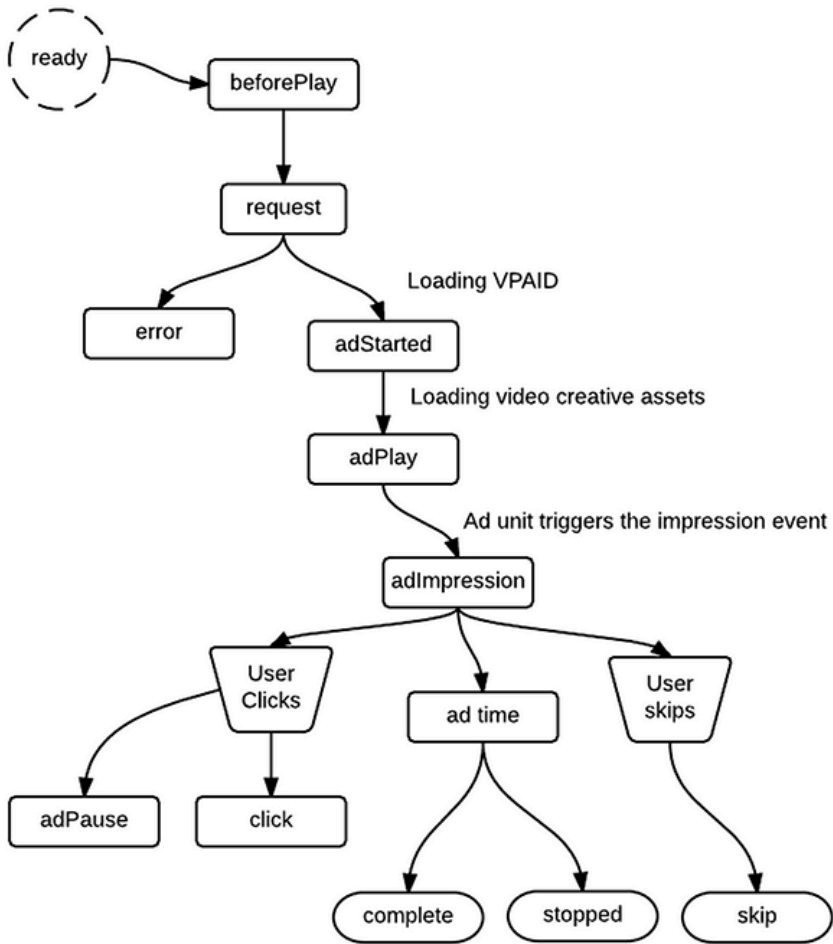
VPAID v2.0 (Section 3.3) định nghĩa một bộ sự kiện chi tiết hơn, bao gồm cả các sự kiện cơ bản từ VAST và các sự kiện bổ sung (từ ad unit đến player và ngược lại).

Event (VPAID)	Nhóm	Sự kiện tương ứng trong VAST	Mô tả
AdLoaded	Ad lifecycle	—	Ad đã tải, sẵn sàng hiển thị.
AdStarted	// (ditto)	creativeView	
AdStopped	//	—	Ad đã được dừng hoàn toàn và tài nguyên đã được giải phóng.
AdSkipped	Playback controls	skip	Ad bị skip.
AdUserClose	//	close	Người dùng đóng ad.
AdSizeChange	Display states	—	Kích thước ad thay đổi.
AdExpandedChange	//	—	Ad chuyển sang trạng thái mở rộng (expanded) hoặc thu gọn (collapsed).
AdUserMinimize	//	collapse	Người dùng thu gọn (minimize ad).
AdSkippableStateChange	Ad states	—	Trạng thái có thể bỏ qua (skippable) thay đổi.
AdDurationChange	//	—	Tổng thời lượng của ad thay đổi (do tương tác,...).
AdLinearChange	//	—	Ad chuyển giữa linear / non-linear mode (ví dụ chuyển thành overlay). Thay đổi cả loại ad, trạng thái và tất nhiên có thể là cả về mặt hiển thị.
AdVolumeChange	Audio controls	Mở rộng mute/unmute của VAST	Thay đổi âm lượng.
AdImpression	Engagement / Interaction	Tương ứng thẻ <Impression> (the VAST element)	User “có thể nhìn thấy” ad. Có nghĩa ad đã render và ít nhất một phần của quảng cáo được hiển thị, có khả năng được người dùng nhìn thấy.
AdClickThru	//	Tương ứng thẻ <ClickTracking> của VAST.	User thực hiện click quảng cáo
AdInteraction	//	—	Người dùng tương tác với ad ngoài click.
AdUserAcceptInvitation	//	acceptInvitation	Người dùng chấp nhận lời mời (invitation).
AdError	Errors & issues	error (code 901 cho VPAID error)	Có lỗi xảy ra.

Ad endpoints

- **Ad break:** một **chuỗi quảng cáo** phát liên tục, có thể chứa một hay nhiều ads (a sequenced group of ads) → events: `ad_break_start` , `ad_break_complete` / `ad_break_end` .
- Nếu một break chứa nhiều ads thì gọi là **ad podding** (được giới thiệu từ IAB VAST 3.0)
- **Ad:** một **quảng cáo đơn lẻ** bên trong ad break → events: `ad_started` , `ad_completed` , `ad_error` , `ad_buffering_started` , `ad_buffering_ended` ...

JW Player (JWP) **hỗ trợ VPAID 2.0**, cụ thể là phiên bản JavaScript (JS-based creatives), bắt đầu từ **phiên bản 7.1**



<https://docs.jwplayer.com/platform/docs/vpaid-20-interactive-ads-reference>

Brightcove phân rõ events thành 2 loại cho riêng: cho quảng cáo đơn lẻ và cho ad podding.

- **ads-pod-started**: ad pod linear bắt đầu.
- **ads-pod-ended**: ad pod linear kết thúc.
- **ads-allpods-completed**: tất cả ad pods đã phát xong.

Event (FPL)	Nhóm	Mô tả
<code>ad_break_started</code> (mandatory)	Ad lifecycle	Báo hiệu bắt đầu phát quảng cáo, có thể là một hay nhiều quảng cáo (ad pod started).
<code>ad_break_ended</code> (mandatory)	// (ditto)	Được gửi khi kết thúc các quảng cáo, ngay trước khi phát lại nội dung chính (video content).
<code>ad_request</code> (mandatory)	//	Gửi request quảng cáo.
<code>ad_manifest_loaded</code> / <code>ad_metadata_loaded</code> (optional)	//	Dùng cho debug / đo performance (đã load ad metadata) → có thể không cần dùng player events + thêm property của ad context .
<code>ad_loaded</code> (mandatory)	//	Ad đã load thành công từ ad server (tức đã load creative / ad data, và phải là trạng thái playback-ready). Lưu ý: rất nhiều SDK/players nhất là HTML5 implement sai event này , báo hiệu khi ad break / ad pod metadata đã sẵn sàng chứ không phải là playback-ready . Lý do các SDK/players ngày dựa thuần túy vào HTML5 <video> nhưng fired ngay khi có event <code>loadedmetadata</code> chứ chưa đến <code>canplay</code> / <code>canplaythrough</code> .
<code>ad_started</code> (mandatory)	//	Ad bắt đầu phát.
<code>ad_paused</code> (optional)	Playback controls	User tạm dừng quảng cáo → khác với content, chỉ cần ghi nhận có pause → thường ad không paused → có thể không cần → dùng chung player events + thêm property của ad context .
<code>ad_resumed</code> (optional)	//	User tiếp tục (sau khi đã tạm dừng) → chỉ cần ghi nhận có resumed → thường ad không paused → có thể không cần → dùng chung player events + thêm property của ad context .
<code>ad_skipped</code> (mandatory)	//	
<code>ad_muted</code> (optional)	Audio controls	→ có thể không cần → dùng chung player events + thêm property của ad context .

Event (FPL)	Nhóm	Mô tả
<code>ad_volume_changed</code> (optional)	//	→ có thể không cần → dùng chung player events + thêm property của ad context .
<code>ad_impression</code> (mandatory)	Engagement / Interaction	Ad hiển thị trên màn hình , user thấy được nhưng không đồng nghĩa với ad video đã bắt đầu phát. Với những loại non-linear / overlay / companion ad, ví dụ là static banner không có sự kiện <code>ad_started</code> .
<code>ad_clicked</code> (mandatory)	//	
<code>ad_progress</code> (mandatory)	Progress / Completion	progress-[current-time] hoặc progress-[0-100]% → có thể không cần → dùng chung heartbeat/playing + playhead + thêm property của ad context .
<code>ad_first_quartile</code> (optional)	//	
<code>ad_midpoint</code> (optional)	//	
<code>ad_third_quartile</code> (optional)	//	
<code>all_ads_completed</code> (optional)	//	
<code>ad_completed</code> (mandatory)	//	
<code>ad_error</code> (mandatory)	Errors & issues	Bất kỳ lỗi nào (VAST error code, timeout, unsupported creative).
<code>ad_autoplay_failed</code> (optional)	//	
<code>ad_playback_failure</code> (mandatory)	//	Creative không play được → quan tâm ad không playback được → chi tiết xem các player error events như streaming error hay media error .
<code>ad_buffering_started</code> / <code>ad_buffering_ended</code> (mandatory)	Performance / QoE metrics	Buffering (startup) → ảnh hưởng mạnh đến first impression → có thể không cần dùng chung player events + thêm property của ad context .
<code>ad_rebuffering_started</code> / <code>ad_rebuffering_ended</code> (mandatory)	// (ditto)	→ có thể không cần dùng chung player events + thêm property của ad context .
<code>ad_fallback</code> (optional)	// (ditto)	SDK fallback sang ad khác. Xem https://support.google.com/admanager/answer/3007370?hl=en

Video QoE metrics



Tham khảo

- MUX video quality metrics <https://www.mux.com/docs/guides/data-video-quality-metric>

Ad metrics



Xem thêm

Conviva https://docs.conviva.com/learning-center-files/content/ei_application/metric_dictionary/metric_dictionary.html
MUX <https://www.mux.com/docs/guides/understand-metric-definitions>